

API Learning Map

Udemy Web Development Course — Section 5 Complete

5.2 JSON

5.3 Axios

5.4 Auth

5.5 REST

5.6 Project



SECTION 5.2

JSON — JavaScript Object Notation

JSON is the universal language of APIs. Data is always sent and received as JSON strings — you convert them into usable JavaScript objects and back.

PARSING JSON (STRING → OBJECT)

```
const json = '{"name":"Chicken Taco","price":2.99}'

// Convert string to JS object
const obj = JSON.parse(json)
console.log(obj.name) //
"Chicken Taco"
console.log(obj.price) // 2.99
```

STRINGIFY + SEARCH

```
// Convert object back to string
JSON.stringify(obj)
// '{"name":"Chicken Taco",...}'

// Search a JSON array
const recipe = data.find(item =>
  item.name.includes("Chicken")
)
```

- ✓ JSON.parse() — converts a string into a usable JavaScript object
- ✓ JSON.stringify() — converts an object back into a string for display
- ✓ Searching arrays with .find() to get a specific item
- ✓ Passing JSON data to EJS templates for rendering

JSON.parse()

JSON.stringify()

.find()

EJS Templates



SECTION 5.3

Axios — Making HTTP Requests

Axios is a library that lets your server talk to external APIs. It handles sending requests and receiving responses using `async/await`.

BASIC GET WITH ASYNC/AWAIT

```
app.get("/", async (req, res) =>
{
  try {
    const response = await
    axios.get(
      "https://bored-
    api.appbrewery.com/random"
    )
    res.render("index.ejs", {
      data: response.data
    })
  } catch (error) {
    res.render("index.ejs", {
      error: error.message
    })
  }
})
```

WITH QUERY PARAMETERS

```
// params builds URL
automatically:
// /filter?
type=social&participants=2
axios.get(URL + "/filter", {
  params: {
    type: req.body.type,
    participants:
    req.body.participants
  }
})

// Pick a random item from
results
const items = response.data
const random = items[
  Math.floor(Math.random() *
items.length)
]
```

Browser



Express Route



Axios



External API



EJS Render

- ✓ `async/await` — cleaner way to handle promises instead of `.then()`
- ✓ `try/catch` — handles errors so the server doesn't crash
- ✓ `params` option — Axios builds query strings automatically
- ✓ `Math.random()` — pick a random item from an array of results

axios.get()

async/await

try/catch

params

response.data



SECTION 5.4

API Authentication — 4 Methods

APIs use authentication to verify who is making a request. Each method sends credentials differently — in the URL, in a header, or encoded in the request.

No Auth

Public APIs — no credentials needed

```
axios.get(API_URL + "random")
```

Basic Auth

Username + password encoded in the header

```
axios.get(URL, {  
  auth: {  
    username: yourUsername,  
    password: yourPassword,  
  }  
})
```

API Key

Unique key passed as a query param in the URL

```
axios.get(URL, {  
  params: {  
    score: 5,  
    apiKey: yourAPIKey,  
  }  
})  
// → /filter?score=5&apiKey=...
```

Bearer Token

Token sent in the Authorization header

```
axios.get(URL, {  
  headers: {  
    Authorization:  
      `Bearer ${yourBearerToken}`  
  }  
})
```

- ✓ No Auth — public endpoints, plain `axios.get()` with no options
- ✓ Basic Auth — `auth: { username, password }` option in Axios
- ✓ API Key — passed as params in the URL query string
- ✓ Bearer Token — passed in headers as `Authorization: Bearer ...`

No Auth

Basic Auth

API Key

Bearer Token



SECTION 5.5

REST APIs — CRUD Operations

REST APIs use 5 HTTP methods to cover every possible data operation. Together they form CRUD — Create, Read, Update, Delete — the backbone of every web app.

METHOD	ACTION	AXIOS CALL	URL PATTERN
GET	Read data	<code>axios.get(url, config)</code>	<code>/secrets/42</code>
POST	Create new	<code>axios.post(url, body, config)</code>	<code>/secrets</code>
PUT	Replace all fields	<code>axios.put(url, body, config)</code>	<code>/secrets/42</code>
PATCH	Update some fields	<code>axios.patch(url, body, config)</code>	<code>/secrets/42</code>
DELETE	Remove data	<code>axios.delete(url, config)</code>	<code>/secrets/42</code>

REUSABLE CONFIG — DEFINE ONCE, USE IN EVERY REQUEST

```
// Define once at the top
const config = { headers: { Authorization: `Bearer ${yourBearerToken}` } }

// Reuse everywhere — no repetition
const result = await axios.get(API_URL + "/secrets/" + id, config)
const result = await axios.put(API_URL + "/secrets/" + id, req.body, config)
const result = await axios.delete(API_URL + "/secrets/" + id, config)
```

- ✓ PUT replaces the entire record — all fields must be sent
- ✓ PATCH updates only the fields you send — others stay unchanged
- ✓ DELETE takes no body — just the ID in the URL
- ✓ config object stores auth and is reused across all requests

GET

POST

PUT

PATCH

DELETE

config



SECTION 5.6

Secrets Project — Built from Scratch

A complete web app built with no scaffolding — combining everything from this section into a working project that fetches real data from a live API.

COMPLETE ROUTE — FETCH & RENDER

```
app.get("/", async (req, res) =>
{
  try {
    const result = await
axios.get(
  API_URL + "/random"
)
    res.render("index.ejs", {
      secret: result.data.secret,
      user:
result.data.username
    })
  } catch (error) {
    res.render("index.ejs", {
      error: error.message
    })
  }
})
```

KEY LESSONS LEARNED

- ✓ req always before res — "request in, response out"
- ✓ Variable names must exactly match what the EJS template uses
- ✓ res.render must be inside try{} to access the result variable
- ✓ ReferenceError = variable name doesn't match template
- ✓ express.static("public") serves CSS and images

localhost:3000



Express Route



axios.get()



secrets-api



res.render()



index.ejs

Express

Axios

EJS

async/await

try/catch

Section 5 Complete

You went from understanding what JSON is to building a full project that talks to real APIs.



Node.js

Runtime environment



Express

Web server & routing



Axios

HTTP requests



EJS

HTML templating